

JVM, MEMORY & GARBAGE COLLECTION

1. What different memory areas does the JVM use while running a program?

The JVM primarily uses the **Heap** to store objects and class instances, which is shared by all threads. It uses the **Stack** to store local variables and manage method execution (one stack per thread). It also utilizes **Metaspace** for storing class metadata and the **Program Counter (PC) Register** to track the current instruction being executed.

2. Why is stack memory quicker to access compared to the heap?

Stack memory is faster because it operates on a strict **LIFO (Last-In, First-Out)** basis, meaning memory allocation and deallocation are just simple pointer movements. In contrast, the Heap requires complex management to find free blocks of memory for new objects and runs garbage collection processes to clean them up.

3. How does Java clean unused objects automatically?

Java uses a background process called the **Garbage Collector (GC)**. It automatically scans the memory to identify objects that are no longer reachable or used by the application. Once identified, it deletes these objects to reclaim memory space for new allocations.

4. What garbage collection techniques does the JVM rely on?

The most common technique is **Mark-and-Sweep**, where the GC "marks" live objects and "sweeps" (deletes) the rest. It also heavily uses **Generational Collection**, which divides the heap into "Young" and "Old" generations. This allows the GC to clean up short-lived objects quickly in the Young generation while checking long-lived objects less frequently.

5. How can a Java program still have memory leaks even with GC running?

A memory leak occurs when an application unintentionally holds references to unused objects, preventing the GC from deleting them. Common causes include **static variables** holding large lists that never get cleared, or unclosed resources like database connections. Since the objects are still "referenced," the GC assumes they are needed and leaves them in memory.

6. Which tools can you use to find and debug memory leak problems?

Developers commonly use **VisualVM** and **JConsole** (included with the JDK) to monitor heap usage in real-time. For detailed analysis, tools like **Eclipse Memory Analyzer (MAT)** or **JProfiler** can inspect "heap dumps" to pinpoint exactly which objects are consuming the most memory and why.

7. What is Metaspace and how is it not the same as PermGen?

PermGen was a fixed-size memory area in older Java versions used for class metadata, which often caused "OutOfMemory" crashes. **Metaspace**, introduced in Java 8, replaces PermGen and uses **native OS memory**. This allows it to grow automatically based on the application's needs, significantly reducing crash risks.

8. How is the Young Generation different from the Old Generation in JVM memory?

The **Young Generation** is where all new objects are initially created; it is small and garbage collected very frequently (Minor GC). The **Old Generation** stores objects that have survived multiple garbage collection cycles in the Young Generation. The Old Gen is larger and is cleaned up less often (Major GC) because it is more "expensive" to scan.

9. Can the garbage collector handle circular object references?

Yes, the JVM's Garbage Collector handles circular references (e.g., Object A references B, and B references A) correctly. It does not rely on simple reference counting. Instead, it checks if objects are reachable from a **GC Root**; if a group of circular objects is isolated from the root, the GC deletes the entire group.

10. How does the use of static variables impact memory?

Static variables belong to the class rather than a specific object instance, so they remain in memory for the entire lifetime of the application. If you use static collections (like a List or Map) and keep adding data without removing it, they act as a major source of memory leaks because the GC is never allowed to clean them up.

11. How does the class-loading process affect memory usage?

Every class loaded by the JVM requires memory in **Metaspace** to store its metadata (bytecode, field names, method names). If an application or framework generates a massive number of dynamic classes at runtime (common in Spring or Hibernate), it can fill up Metaspace and cause memory issues.

12. Is it possible for the JVM to remove a loaded class?

Yes, a loaded class can be removed (unloaded), but only if the **ClassLoader** that loaded it is garbage collected and no instances of that class exist. This is relatively rare in standard applications but is a critical feature for web servers that need to reload applications without restarting the JVM.

13. What do we mean by a GC Root in memory management?

GC Roots are the starting points for the garbage collection process; they are objects that are guaranteed to be "alive." Examples include active **Threads**, **local variables** on the stack, and **static variables**. The GC traces all references starting from these roots to find out which objects are still in use.

14. What causes a Stop-The-World pause and what happens during it?

A **Stop-The-World** pause is a state where the JVM halts all application threads to perform garbage collection safely. During this pause, the application becomes completely unresponsive. While Minor GCs cause negligible pauses, a Full GC can cause noticeable freezes if the heap is very large.

15. How do Minor GC, Major GC, and Full GC differ from each other?

Minor GC cleans only the Young Generation and is extremely fast. **Major GC** cleans the Old Generation and is usually slower. **Full GC** cleans the entire Heap (Young + Old) and typically Metaspace as well; it is the most expensive operation and causes the longest system pauses.

16. What is escape analysis and why does the JVM use it?

Escape Analysis allows the JVM to determine if a new object is used only within a single method and never "escapes" to other threads. If an object stays local, the JVM may optimize

by allocating it on the **Stack** instead of the Heap. This improves performance because stack memory is instantly freed when the method ends, avoiding GC overhead.

17. What is a TLAB (Thread Local Allocation Buffer)?

A **TLAB** is a small chunk of the Heap reserved exclusively for a single thread to create new objects. It allows threads to allocate memory quickly without having to lock the entire Heap or compete with other threads. This significantly speeds up object creation in multi-threaded applications.

18. Which JVM flags have you used to set memory size or change GC behavior?

This is a personal experience question, but a standard answer is: Common flags include `-Xms` to set the initial heap size and `-Xmx` to set the maximum heap size. To tune Garbage Collection, flags like `-XX:+UseG1GC` or `-XX:+UseZGC` are used to select specific algorithms that minimize pause times.

19. How does the JVM decide the right time to run garbage collection?

The JVM triggers garbage collection automatically when the **Young Generation gets full**, which initiates a Minor GC. Major or Full GCs are triggered based on complex heuristics, such as when the Old Generation exceeds a certain percentage of capacity or when the system detects memory pressure.

20. What is reference counting and why doesn't the JVM depend on it?

Reference counting tracks how many links point to an object, deleting it when the count hits zero. The JVM avoids this technique because it fails to detect **circular references** (where isolated objects point to each other). Instead, Java uses "Reachability Analysis" from GC Roots to ensure all truly unused objects are found.

CLASSLOADERS, REFLECTION & ANNOTATIONS

1. What are the main parts that make up JVM architecture?

The JVM is composed of three main subsystems: the **ClassLoader Subsystem** (for loading class files), the **Runtime Data Areas** (memory like Heap and Stack), and the **Execution Engine** (which includes the Interpreter, JIT Compiler, and Garbage Collector). It also includes the **Native Method Interface (JNI)** to interact with native libraries.

2. What role does a ClassLoader play in Java?

A **ClassLoader** is responsible for dynamically loading Java class files (.class) into the JVM memory at runtime. It follows a "lazy loading" model, meaning it does not load classes until the application specifically references or needs them. Without it, the JVM would not know how to convert your bytecode into running code.

3. What are the different types of ClassLoaders in Java?

There are three primary built-in loaders:

1. **Bootstrap ClassLoader:** Loads core Java libraries (like java.lang.*) from the JDK.
2. **Extension (Platform) ClassLoader:** Loads standard extensions from the lib/ext directory.
3. **Application (System) ClassLoader:** Loads classes from your application's classpath.

4. How is Class.forName() different from ClassLoader.loadClass()?

Class.forName() loads the class and immediately **initializes** it, meaning it executes the static blocks of the class. **ClassLoader.loadClass()** only loads the class definition into memory but does **not** initialize it until the class is actually used. You use **forName()** when you need the side effects of static initialization (like registering a JDBC driver).

5. What is reflection and where is it usually used?

Reflection is an API that allows a program to inspect and modify the behavior of classes, methods, and fields at runtime, even if they are private. It is heavily used in **frameworks** like

Spring, Hibernate, and JUnit to create objects, inject dependencies, and run tests dynamically without knowing the class names at compile time.

6. What is the Reflection API mainly used for?

It is mainly used for building **generic frameworks** and tools. IDEs use it for auto-completion, debuggers use it to inspect variables, and frameworks use it for dependency injection and object-relational mapping (ORM). It enables software to manipulate code structures dynamically.

7. Which Java package handles regular expression (regex) operations?

The **java.util.regex** package handles regex operations in Java. It primarily consists of the **Pattern** class (to compile the regex) and the **Matcher** class (to perform matching operations against text).

8. What does the @Retention annotation define?

The **@Retention** annotation specifies the **lifecycle** of an annotation, how long it should be kept around. It determines whether the annotation is discarded during compilation, stored in the class file, or made available at runtime for reflection.

9. What does the @Target annotation specify?

The **@Target** annotation specifies **where** an annotation can be applied in your code. It restricts usage to specific elements, such as methods (**ElementType.METHOD**), fields (**ElementType.FIELD**), classes (**ElementType.TYPE**), or parameters (**ElementType.PARAMETER**).

10. How does the Parent Delegation Model work for class loading?

When a **ClassLoader** is asked to load a class, it delegates the request to its **parent** **ClassLoader** first before trying to load it itself. This ensures security and stability; for example, it prevents a malicious application from replacing core classes like `java.lang.String` with a custom version.

11. Is it possible to break the delegation model? How?

Yes, it is possible to break the delegation model by creating a custom `ClassLoader` and overriding the `loadClass()` method to bypass the parent call. Web servers like **Tomcat** often do this to isolate different web applications so they can use different versions of the same library without conflict.

12. How do you write your own custom ClassLoader?

To create a custom `ClassLoader`, you extend the `java.lang.ClassLoader` class. You typically override the `findClass()` method to define your custom logic for reading the byte array from a file or network and then call `defineClass()` to convert those bytes into a generic Java Class.

13. What problems or dangers can reflection introduce?

Reflection bypasses compile-time type checking, making code harder to debug and maintain. It is also significantly **slower** than direct method calls because the JVM cannot optimize it easily. Furthermore, it creates **security risks** by allowing access to restricted internals of a class.

14. Why is reflection considered to break encapsulation rules?

Reflection allows you to call `setAccessible(true)` on **private** fields and methods, giving you access to internal implementation details that were meant to be hidden. This breaks the core OOP principle of encapsulation, which dictates that an object should control its own state and hide its internals from the outside world.

15. What is meant by annotation processing?

Annotation processing is a compile-time hook that allows libraries to scan for annotations and generate additional source code or files automatically. Tools like **Lombok** use this to auto-generate getters, setters, and constructors during compilation, saving developers from writing boilerplate code.

16. How do Source, Class, and Runtime retention levels differ?

- **Source:** The annotation is used by the compiler but discarded immediately (e.g., `@Override`); it does not exist in the compiled `.class` file.

- **Class:** The annotation is stored in the .class file but ignored by the JVM at runtime (this is the default behavior).
- **Runtime:** The annotation is stored in the .class file and is available to the JVM via **reflection**, making it useful for frameworks like Spring.

COLLECTION INTERNALS & PERFORMANCE

1. How does HashMap store and retrieve key–value pairs internally?

HashMap uses an array of "buckets" to store data. When you add a pair, it calculates the key's **hash code** to determine the specific index (bucket) where the data belongs. To retrieve it later, it recalculates the hash to jump directly to that bucket and finds the value, making retrieval extremely fast.

2. What does HashMap do when two keys end up with the same hash value?

This is called a **hash collision**. When two keys map to the same bucket, HashMap stores them in a **Linked List** (or a Tree in Java 8+) within that single bucket. It then uses the `.equals()` method to compare the actual key objects and find the correct one within that list.

3. What updates were introduced to HashMap in Java 8?

The most significant performance update was handling high collision rates. If a single bucket accumulates more than 8 items, Java 8 automatically converts that bucket's internal structure from a **Linked List** to a **Red-Black Tree**. This improves the worst-case search speed from $O(n)$ to $O(\log n)$.

4. Can you use a custom class as a key in a HashMap?

Yes, you can use a custom class, but you **must** correctly override both `equals()` and `hashCode()`. If you don't, the HashMap will use the object's memory address for hashing, meaning you won't be able to retrieve your data using a logically equivalent object later.

5. How is ConcurrentHashMap structured internally?

In Java 8 and later, ConcurrentHashMap uses an array of nodes (buckets) similar to HashMap. However, it does not lock the entire map for updates; instead, it locks only the specific **bucket**

(node) being modified using synchronized. It also utilizes **CAS (Compare-And-Swap)** operations for safe, lock-free insertions when a bucket is empty.

6. How does ConcurrentHashMap improve performance in multi-threading?

It allows multiple threads to **read** data simultaneously without any locking (non-blocking reads), which is very fast. For **writing**, it uses fine-grained locking (locking only one bucket out of many), so multiple threads can write to different parts of the map at the exact same time without waiting for each other.

7. How is Hashtable different from ConcurrentHashMap?

Hashtable is an old legacy class that locks the **entire map** for every write operation, creating a bottleneck where only one thread can work at a time. **ConcurrentHashMap** is modern and uses bucket-level locking, allowing for much higher concurrency and throughput.

8. What are the expected time complexities of HashMap, HashSet, TreeMap, and TreeSet?

HashMap and HashSet offer **O(1)** (constant time) performance for adding and retrieving items on average. TreeMap and TreeSet offer **O(log n)** performance because they must navigate a tree structure to keep elements in sorted order.

9. What internal data structures are used by HashMap, HashSet, TreeMap, and TreeSet?

HashMap and HashSet use an **array of buckets** combined with Linked Lists (or Red-Black Trees for collisions). TreeMap and TreeSet internally use a **Red-Black Tree** (a self-balancing binary search tree). Note that HashSet is essentially just a wrapper around a HashMap.

10. How does TreeSet keep its elements sorted?

TreeSet maintains order by inserting elements into a **Red-Black Tree**. As every new item is added, the tree automatically re-balances itself to ensure the correct sorting order (ascending by default). It relies on the object's `compareTo()` method or a custom `Comparator` to decide where the new node fits.

11. How does LinkedHashMap preserve insertion order?

LinkedHashMap extends standard HashMap but adds a **doubly-linked list** running through all its entries. Each entry keeps a pointer to the element added before it and the element added after it. This allows you to iterate through the map in the exact sequence the items were inserted.

12. What does "access-order" mean in LinkedHashMap and how is it useful for LRU Cache?

Access-order means the map moves an entry to the end of the list every time it is accessed (read or updated). This is perfect for building an **LRU (Least Recently Used) Cache** because the "oldest" unused items naturally drift to the front of the list, where they can be easily identified and removed.

13. Which tree-balancing method does TreeMap use?

TreeMap uses the **Red-Black Tree** algorithm. This is a self-balancing binary search tree that ensures the tree remains relatively flat (balanced). It guarantees that operations remain efficient ($O(\log n)$) even if you insert sorted data that would normally make a standard tree lopsided.

14. What does HashMap do when there are too many collisions?

As mentioned in the Java 8 updates, if a bucket exceeds a specific threshold (`TREEIFY_THRESHOLD = 8`), the HashMap transforms that bucket's linked list into a **balanced tree**. This prevents "Hash DoS" attacks or poor performance where looking up a key becomes very slow due to a long list.

15. Why does ConcurrentHashMap not allow null values or null keys?

It disallows nulls to avoid **ambiguity** in concurrent scenarios. If `map.get(key)` returned null, you wouldn't know if the key was missing or if the value was actually null. In a multi-threaded environment, you cannot safely check `containsKey()` afterwards because the map might have changed in the milliseconds between the two calls.

16. Why is HashMap unsafe to use in multi-threaded programs?

HashMap has no synchronization mechanisms. If multiple threads modify the map structure (like resizing the array) simultaneously, it can lead to data loss, corruption, or (in older versions) infinite loops. The map's internal state becomes inconsistent, causing the application to crash or behave unpredictably.

17. How does CopyOnWriteArrayList work while iterating?

When you iterate over a CopyOnWriteArrayList, the iterator looks at a "snapshot" of the array taken when the loop started. If another thread adds or removes elements, it creates a **brand new copy** of the underlying array for the modification. The iterator continues safely reading the old array, effectively ignoring the changes.

18. What is the difference between fail-fast and fail-safe behavior?

Fail-fast iterators (like HashMap) throw a ConcurrentModificationException instantly if they detect changes to the collection during iteration. **Fail-safe** iterators (like ConcurrentHashMap or CopyOnWriteArrayList) allow modifications during iteration because they work on a consistent view of the data (often using weak consistency or snapshots).

CONCURRENCY & EXECUTOR FRAMEWORK

1. What are the basics of the Java Memory Model?

The **Java Memory Model (JMM)** defines how threads interact with memory. It specifies that every thread has its own local "**working memory**" (cache) where it stores copies of variables from the shared "**main memory**." The JMM dictates rules for when a thread must write changes back to main memory so other threads can see them.

2. What is the visibility issue between threads?

A **visibility issue** occurs when one thread modifies a shared variable in its local cache, but hasn't yet written the change back to main memory. Other threads continue to read the "stale" (old) value from their own caches. To fix this, we use keywords like volatile or synchronized to force memory updates.

3. How is ReentrantLock different from synchronized?

ReentrantLock is an explicit lock object that gives you more control than the implicit **synchronized** keyword. It allows advanced features like **fairness** (giving the lock to the longest-waiting thread) and the ability to try locking without waiting forever (**tryLock()**). However, unlike **synchronized**, you must manually release the lock in a **finally** block.

4. What does ExecutorService help you with?

ExecutorService is a framework that manages a pool of worker threads for you. Instead of manually creating and starting a new **Thread** for every task (which is expensive), you simply submit tasks to the service. It reuses existing threads efficiently and handles the lifecycle of task execution.

5. How does submit() differ from execute()?

execute() is used for **Runnable** tasks where you don't care about the result; it is a "fire-and-forget" method. **submit()** can take either **Runnable** or **Callable** tasks and returns a **Future** object. You can use this **Future** to check if the task is done or to retrieve the returned result later.

6. What is a RejectedExecutionHandler used for?

This handler decides what to do when a thread pool cannot accept any more tasks (e.g., the queue is full and all threads are busy). Common strategies include throwing an exception (**AbortPolicy**), running the task on the caller's thread (**CallerRunsPolicy**), or quietly discarding the oldest task (**DiscardOldestPolicy**).

7. How does ThreadPoolExecutor work inside?

Internally, it manages a **core pool** of threads that stay alive and a **work queue** for holding pending tasks. If the queue fills up, it can create extra threads up to a **maximum pool size**. If the max size is reached and the queue is full, it triggers the **RejectedExecutionHandler**.

8. What are common challenges when working with threads?

The three biggest challenges are **Race Conditions** (data corruption due to bad timing), **Deadlocks** (threads waiting on each other forever), and **Memory Consistency Errors** (threads

seeing old data). Debugging these issues is difficult because they are often non-deterministic (they don't happen every time).

9. How do you handle a thread interruption?

When a thread is interrupted, it typically throws an `InterruptedException`. You should catch this exception and stop what you are doing (e.g., break a loop). It is also good practice to call `Thread.currentThread().interrupt()` inside the catch block to preserve the interruption status for higher-level methods.

10. How can we check if a thread currently owns a lock?

You can check if the current thread holds the monitor lock of a specific object by using the static method `Thread.holdsLock(Object obj)`. This returns true if the current thread is inside a synchronized block or method for that specific object.

11. When should `ThreadLocal` be used?

`ThreadLocal` should be used when you need to store data that is specific to a single thread and should not be shared. A common use case is storing a **User ID** or **Transaction Context** in a web server request; the data stays available throughout that thread's execution without passing it as a parameter to every method.

12. What happens if an exception is thrown inside a synchronized block?

If an unhandled exception occurs inside a synchronized block, the JVM automatically **releases the lock**. This safety mechanism prevents the system from entering a permanent deadlock state where a crashed thread holds onto a lock forever.

13. What is the difference between `Runnable` and `Callable`?

Runnable is the older interface; its `run()` method returns void and cannot throw checked exceptions. **Callable** is the newer, more flexible interface; its `call()` method returns a result (via a Generic type) and is allowed to throw checked exceptions.

14. What does “happens-before” mean in concurrency?

"**Happens-before**" is a guarantee provided by the Java Memory Model. If Action A "happens-before" Action B, then all changes made in A are visible to B. For example, releasing a lock *happens-before* acquiring that same lock, ensuring the second thread sees everything the first thread did.

15. How does volatile guarantee visibility?

The volatile keyword forces variable reads and writes to go directly to **main memory**, bypassing CPU caches. This guarantees that if Thread A changes a volatile variable, Thread B will immediately see the updated value. However, it does *not* provide atomicity for compound actions like `i++`.

16. How does volatile differ from synchronized?

Volatile is a lightweight synchronization mechanism that only ensures **visibility** of variables. **Synchronized** ensures both **visibility** and **atomicity** (exclusive access). You use volatile for simple flags, but you must use synchronized (or locks) for complex logic where multiple steps need to happen as a single unit.

17. What are atomic classes and why do we use them?

Atomic classes (like `AtomicInteger`, `AtomicBoolean`) provide a way to update variables in a thread-safe, lock-free manner. They use efficient hardware-level instructions (CAS) to ensure safety without the performance penalty of using the heavier synchronized keyword.

18. What is CAS (Compare and Swap) and where is it used?

CAS is an optimistic locking technique used in low-level concurrency. It says: "I think the current value is A; if it is, change it to B. If it's not A, tell me, and I'll try again." It is the foundation of Java's **Atomic classes** and **ConcurrentHashMap**.

19. What is a deadlock?

A **deadlock** occurs when two threads are stuck waiting for each other to release a lock. For instance, Thread 1 holds Lock A and wants Lock B, while Thread 2 holds Lock B and wants Lock A. Neither can proceed, causing the application to hang.

20. What is a livelock?

A **livelock** is similar to a deadlock, but the threads are technically still running (not blocked). They are constantly changing state in response to each other (like two people meeting in a hallway and stepping left/right at the same time) but never actually making progress on their task.

21. What is starvation in multi-threading?

Starvation happens when a thread is perpetually denied access to necessary resources (like CPU time or locks). This usually occurs when high-priority threads constantly occupy the resources, leaving lower-priority threads waiting indefinitely.

22. What is ForkJoinPool used for?

ForkJoinPool is a specialized thread pool designed for "divide and conquer" algorithms. It uses a **work-stealing** algorithm: if a worker thread runs out of tasks, it can "steal" work from the queue of another busy thread. This makes it extremely efficient for recursive tasks like processing large arrays.

23. How does CompletableFuture work internally?

CompletableFuture represents a future result of an asynchronous computation that can be manually completed. It allows you to chain tasks together (e.g., "do X, then do Y") using non-blocking callbacks. Internally, it uses a stack of dependent actions and typically runs them on the `ForkJoinPool.commonPool()`.

24. How does lock striping improve performance in ConcurrentHashMap?

Lock striping splits the map into several independent segments (buckets), each with its own lock. Instead of locking the entire map for every write operation, a thread only locks the specific segment it needs. This allows multiple threads to write to different parts of the map simultaneously, drastically improving throughput.

ADVANCED STREAMS & LAMBDA

1. How is filter() different from map() in streams?

filter() is a boolean check that decides whether to keep an element or discard it; the resulting stream may have fewer elements than the original. **map()** is a transformation method that converts every element into something else (like changing a String to an Integer); the resulting stream always has the same number of elements as the original.

2. What is the difference between intermediate and terminal operations?

Intermediate operations (like filter or map) are lazy; they return a new Stream and do not execute immediately. **Terminal operations** (like collect, forEach, or count) trigger the actual processing of the pipeline and return a final result or void, effectively closing the stream.

3. How is Stream.sorted() different from Collections.sort()?

Stream.sorted() produces a *new* stream containing the sorted elements, leaving the original data source unchanged. **Collections.sort()** modifies the original list **in-place**, permanently changing the order of elements in that collection.

4. How do parallel streams work internally?

Parallel streams use the **ForkJoinPool** to recursively split the data into smaller chunks using a **Splitter**. These chunks are processed simultaneously on multiple threads. Finally, the results from all threads are combined (joined) to form the final output.

5. What is the real difference between map() and flatMap()?

map() performs a one-to-one transformation: one input element produces exactly one output element (e.g., List<String> becomes List<Integer>). **flatMap()** performs a one-to-many transformation: it expects the function to return a *Stream* for each element, and then it "flattens" all those streams into a single continuous stream (e.g., flattening a List<List<String>> into a List<String>).

6. How is `peek()` different from `map()`, and why can `peek` be risky?

`peek()` is intended mainly for debugging; it performs an action (like printing) without modifying the data and passes the original stream forward. `map()` is for transforming data. `peek()` can be risky because if the stream pipeline is optimized or if no terminal operation is called, the side effect might never happen, leading to confusing bugs.

7. How are `Optional.of()` and `Optional.ofNullable()` different?

`Optional.of(value)` expects a non-null value; if you pass null to it, it throws a `NullPointerException` immediately. `Optional.ofNullable(value)` is safer; if you pass null, it gracefully returns an empty `Optional` (`Optional.empty()`) instead of crashing.

8. How does `skip()` differ from `limit()`?

`skip(n)` ignores the first `n` elements of the stream and passes the remainder down the pipeline. `limit(n)` allows only the first `n` elements to pass through and stops processing the rest. They are often used together for paging data (e.g., "page 2" is `skip(10).limit(10)`).

9. What is the difference between `reduce()`, `sum()`, and `count()`?

`reduce()` is a general-purpose method that combines all elements into a single result using a custom logic you provide (like multiplying all numbers). `sum()` and `count()` are specialized, pre-defined reduction operations specifically for adding numbers or counting total elements, making the code more readable for those common tasks.

10. How does `sorted()` work internally when a comparator is given?

When you provide a **Comparator**, `sorted()` becomes a **stateful intermediate operation**. It must buffer all elements of the stream first to compare them against each other before it can send even a single element to the next step. This can be a performance bottleneck for very large streams.

11. Why are parallel streams dangerous when using shared mutable data?

If your stream operations modify a shared variable (like adding to a standard `ArrayList`) without synchronization, multiple threads will try to update it simultaneously. This leads to

race conditions and data corruption. Parallel streams require stateless, non-interfering functions to work safely.

12. How does a Splititerator work behind the scenes?

A **Splititerator** (Splitable Iterator) is the engine behind parallel streams. Unlike a normal Iterator that just traverses elements, a Splititerator can partition (split) its elements into two parts: one for the current thread to process and another for a new thread to process. This enables efficient work distribution.

13. What is short-circuiting in streams?

Short-circuiting operations stop processing the stream as soon as a result is determined, rather than processing the entire dataset. For example, `findFirst()` stops after finding one item, and `anyMatch()` stops as soon as it finds a single true condition. This provides significant performance gains on large or infinite streams.

14. What is the difference between `collect()` and `reduce()`?

`reduce()` aggregates elements immutably (e.g., summing integers $1 + 2 = 3$). **`collect()`** aggregates elements into a **mutable container** (like adding items into an `ArrayList` or `StringBuilder`). `collect()` is generally more efficient for grouping or listing data because it modifies an existing container rather than creating a new value at every step.

15. Why can't we reuse the same stream again?

A Stream in Java is a **one-time-use** object used to move data, not store it. Once a terminal operation is called, the stream is considered "consumed" or closed. Trying to call another method on it will throw an `IllegalStateException`.

16. What is lazy evaluation in streams?

Lazy evaluation means that intermediate operations (like `filter` or `map`) are not executed immediately when they are defined. Instead, they are recorded and fused together into a single pass that runs only when a **terminal operation** is invoked. This avoids unnecessary loops and temporary object creation.

17. Why do parallel streams rely on ForkJoinPool?

The **ForkJoinPool** uses a "work-stealing" algorithm that is highly efficient for CPU-intensive tasks. By using the common ForkJoinPool, parallel streams can utilize existing worker threads rather than creating expensive new threads for every task. This minimizes overhead while maximizing CPU core usage.

DESIGN PATTERNS

1. Why is the Builder pattern better than having many constructors?

The Builder pattern solves the "telescoping constructor" problem, where classes have multiple constructors with long, confusing lists of parameters. Instead of passing null for optional fields or mixing up argument order, the Builder lets you set properties step-by-step using readable methods (e.g., `.setName("John")`). This results in code that is much cleaner and less error-prone.

2. How is the Builder pattern different from the Factory pattern?

The **Factory Pattern** is used to create an entire object in a single step, often focusing on *which* subclass to instantiate (polymorphism). The **Builder Pattern** is used to construct a complex object step-by-step, focusing on *how* to assemble different parts. Factory is about "what" to create; Builder is about "how" to build it.

3. When should Strategy and State patterns be used?

Use the **Strategy Pattern** when you want to switch between different algorithms (like different sorting or payment methods) for a task at runtime. Use the **State Pattern** when an object's behavior needs to change completely based on its internal state (like a vending machine behaving differently if it has money vs. is empty).

4. How is the Observer pattern used in event-based systems?

The **Observer Pattern** defines a one-to-many relationship where one object (the Subject) notifies a list of other objects (Observers) whenever its state changes. It is the foundation of event handling systems; for example, when you click a button, all registered "listeners" are automatically notified to run their code.

5. What is the difference between REST and RESTful?

REST (Representational State Transfer) is the set of architectural constraints and theories for designing networked applications. **RESTful** describes a specific web service or API that correctly implements and follows those REST principles. Essentially, REST is the theory, and RESTful is the practice.

6. What problem does Dependency Injection solve?

Dependency Injection solves the problem of **tight coupling**, where classes allow themselves to create their own dependencies directly. By injecting dependencies from the outside (like passing a Service into a Controller), classes become independent and easier to test. It allows you to swap real objects for mock objects during unit testing easily.

7. How does the Template Method pattern work?

The **Template Method** defines the skeleton (outline) of an algorithm in a parent class but lets subclasses override specific steps without changing the overall structure. For example, a `DataProcessor` parent class might have a fixed `process()` method that calls abstract `readData()` and `writeData()` methods, which child classes implement differently.

8. How does the Adapter pattern help in real projects?

The **Adapter Pattern** acts as a bridge between two incompatible interfaces, allowing them to work together. It "wraps" an existing class and translates its interface into the one the client expects. This is frequently used when integrating modern code with older, legacy third-party libraries that you cannot change.

9. When should the Decorator pattern be used?

Use the **Decorator Pattern** when you want to add extra responsibilities or features to an individual object dynamically without affecting other objects of the same class. It wraps the original object to provide new behavior, such as adding a scrollbar to a window object or adding milk to a coffee object. It is a flexible alternative to creating a complex hierarchy of subclasses.

10. When is the Proxy pattern helpful?

The **Proxy Pattern** provides a placeholder or representative for another object to control access to it. It is extremely helpful for **lazy loading** (loading a heavy object like a large image only when it is actually requested) or for security (checking user permissions before allowing access to the real object).

11. What does the Chain of Responsibility pattern do?

This pattern passes a request along a chain of potential handlers. Each handler in the chain decides either to process the request or to pass it to the next handler. It is commonly used in logging frameworks (filtering logs by level) or security filters where multiple checks need to happen in a specific sequence.

ADVANCED SERIALIZATION

1. What does serialization mean in Java?

Serialization is the process of converting a Java object's state into a byte stream (a sequence of 0s and 1s). This allows the object to be saved to a file, stored in a database, or sent over a network to another computer. The reverse process, where the byte stream is converted back into a live Java object, is called **deserialization**.

2. Why is serialVersionUID required?

serialVersionUID acts as a unique version identifier for a Serializable class. It ensures that the class definition used to serialize the object matches the class definition used to deserialize it. If you don't define it, Java calculates one automatically, but this can cause errors if you make even small changes to the code.

3. What happens if serialVersionUID changes in future versions?

If the serialVersionUID of the saved object differs from the ID of the class currently in the JVM, Java assumes the versions are incompatible. It will throw an `InvalidClassException` and refuse to load the object. This is why it is best practice to define a fixed serialVersionUID manually to allow for backward compatibility.

4. How does the transient keyword prevent a field from being saved?

When you mark a variable as **transient**, the JVM strictly ignores it during the serialization process. Its value is not written to the byte stream. When the object is deserialized later, that field will not have its original value but will be reset to the default value (like null for objects or 0 for numbers).

5. What if a class contains a non-serializable field?

If a Serializable class contains a field referencing a non-serializable object, Java will throw a `NotSerializableException` at runtime. The entire serialization process will fail. To fix this, that specific field must be marked as transient so the JVM knows to skip it.

6. How do writeObject() and readObject() work?

These are private methods you can define in your class to override the default serialization logic. `writeObject()` lets you manually control what data gets written to the stream (e.g., encrypting a password before saving). `readObject()` lets you control how to read it back and verify or decrypt the data.

7. Can static variables be serialized?

No, **static variables** are not serialized because they belong to the **class** itself, not to any specific object instance. Serialization only saves the state (instance variables) of an individual object. When deserialized, the static variable will simply have whatever value is currently set in the class on that specific machine.

8. How do you serialize objects that refer to each other?

Java automatically handles "object graphs," which are webs of objects that reference each other. It recursively serializes every object linked to the main object you are saving. It also detects circular references (A points to B, B points to A) and handles them correctly so the program doesn't get stuck in an infinite loop.

9. What happens if an exception occurs during serialization?

If an error occurs (like encountering a non-serializable object), the process stops immediately and throws an `IOException` (specifically `NotSerializableException`). The byte stream created up to that point is typically incomplete and corrupt. You must handle the exception to prevent the application from crashing.

10. What is the fix if a class contains a non-serializable member?

You have two main options: either modify the member class to implement `Serializable` as well, or mark the field as `transient`. If you mark it `transient`, you might need to implement logic in `readObject()` to manually re-initialize that data when the object is loaded back.

11. How is `Externalizable` different from `Serializable`?

`Serializable` is automatic; the JVM decides what to save, which is easy but can be slow. **`Externalizable`** gives you complete control but requires you to write the logic yourself in `writeExternal()` and `readExternal()` methods. `Externalizable` is generally faster because you only save the exact data you need.

12. How can serialization break the Singleton pattern?

Deserialization creates a **new instance** of a class from the byte stream, completely bypassing the private constructor. This means you end up with two instances: the original Singleton and the new deserialized copy. This violates the core rule that a Singleton must have only one instance.

13. How do you protect against deserialization attacks?

You should strictly avoid deserializing data from untrusted sources or networks. To protect the application, you can use an **`ObjectInputFilter`** (introduced in newer Java versions) to check classes before loading them. This acts as an allowlist, rejecting any unexpected or malicious classes before they execute code.

14. What do `readResolve()` and `writeReplace()` do?

`readResolve()` is a hook used during deserialization to replace the newly created object with an existing one (this is the standard fix for the **Singleton** issue). `writeReplace()` is a hook used during serialization to swap the object being saved with a different object (often a proxy) before it is written to the stream.

15. What security problems exist with Java's serialization system?

It is a major security risk because deserialization effectively executes code (constructors and methods) just by reading data. Attackers can send a malicious byte stream (called a "gadget chain") that forces the server to execute harmful commands. This is a common vector for **Remote Code Execution (RCE)** attacks.

JAVA Interview Buddy